

PRODUCTION-GRADE PROJECTS

Retail Data Warehouse & Sales Analytics Platform

The Complete Beginner's Architecture & Interview Blueprint

An end-to-end data analytics platform designed to simulate modern enterprise data pipelines. Exposes core Star Schema data modeling, modular Python ETL, Pydantic data quality validation, PostgreSQL performance tuning (range partitioning, index strategies), stored procedures, and a FastAPI REST layer serving clean KPI aggregates.

TARGET INTERVIEW POSITIONS:

- Junior / Associate Data Engineer
- Junior Data Analyst / BI Developer
- SQL Developer / Database Analyst

TECHNICAL STACK COVERED:

Python 3.12, PostgreSQL 16, SQLAlchemy 2.0, FastAPI, Pandas, Docker Compose, Airflow, Pytest

1. Project Architecture & Ingestion Flow

Welcome to the Retail Data Warehouse & Sales Analytics Platform! As a beginner, it is crucial to understand that this project mimics exactly what top-tier firms do to manage their retail BI workloads. It is built in a modular fashion to isolate each concern: data ingestion, cleansing, model validation, SQL warehouse transformations, and API exposition.

The Data Journey: End-to-End Pipeline Stages

The project handles transactions chronologically. Think of it as an assembly line where raw, messy data comes in, gets scrubbed, labeled, double-checked, and finally stored in clean tables.

- * **1. Extraction:** The DataExtractor reads the raw sales CSV, automatically detects file encoding using the chardet library, validates that all expected columns exist, and logs extraction statistics.
- * **2. Raw Validation:** Before cleaning, we run the raw dataframe through the DataValidator. It runs null checks and checks for duplicate PKs to see if the source dataset is healthy.
- * **3. Cleansing:** The DataCleaner parses numeric types, strips blank spaces, drops rows missing critical keys, and enforces boundaries (e.g. discards negative counts or values).
- * **4. Transformation:** The DataTransformer normalizes strings to Title Case, computes Net Revenue ($\text{Sales} * (1 - \text{Discount})$), calculates Net Profit Margin, and identifies statistical outliers using the Interquartile Range (IQR) method.
- * **5. Loading:** The DataLoader executes the Slowly Changing Dimension Type 2 (SCD-2) check on customer records, resolves surrogate dimension keys, and batch-loads the results into target partitions.

2. What is What? Detailed Directory Directory Map

Here is the map of files. Use this guide to easily navigate the codebase during an interview or when working on features.

Core Application Layer (src/)

- * **src/config/config.py:** Uses Pydantic BaseSettings to read environment parameters from the .env file. Centralizes database URLs, file paths, and environment settings.
- * **src/database/database.py:** Initializes the SQLAlchemy engine with a connection pool (10 permanent connections, 20 max overflow) and connection pre-pings to verify active sessions.
- * **src/models/:** Contains the ORM templates (dim_customer, dim_product, dim_region, dim_date, dim_category, fact_sales) mapping Python classes to PostgreSQL physical tables.
- * **src/etl/:** Extract, Clean, Transform, Load logic scripts. Coordinates data processing steps and surrogate key mapping.
- * **src/validation/validator.py:** Custom data quality validation rules checking nulls, duplicate keys, numeric ranges, and business constraints. Generates a weighted percentage quality score.
- * **src/api/:** Contains the FastAPI framework and routes (kpis, sales, products, customers, dashboard) serving JSON data aggregates.

Database Schema & Scripts (sql/ & scripts/)

* **sql/tables/create_dimensions.sql**: DDL script outlining A4 dimension tables, primary keys, defaults, and table comments.

* **sql/tables/create_facts.sql**: Creates the central fact_sales table, setting up database check constraints, cascading deletes, and range partitioning by year.

* **sql/views/**: Analytical SQL queries creating views for YTD sales, regional margins, RFM customer clusters, and monthly trends.

* **sql/stored_procedures/**: PL/pgSQL stored procedures updating materialized views (sp_monthly_refresh) and generating filterable business summaries (sp_sales_summary).

* **scripts/generate_data.py**: Synthetic retail order generator that skews order dates to simulate Q4 holiday seasonality, price thresholds, and data anomalies.

3. Core Analytical Concepts & Code implementation

Slowly Changing Dimension (SCD) Type 2

A Slowly Changing Dimension tracks historical records when an attribute changes. In our project, if customer segment shifts from 'Consumer' to 'Corporate', we do not overwrite the segment. Instead, we keep history by marking the old row inactive and inserting a new current active row.

```
# SCD Type 2 Logic in src/etl/load/loader.py
if existing.segment != segment:
    # 1. Expire existing record
    existing.is_current = False
    existing.expiry_date = order_date - datetime.timedelta(days=1)

    # 2. Insert new current active record
    new_customer = DimCustomer(
        customer_id=customer_id, customer_name=cust_name,
        segment=segment, effective_date=order_date,
        expiry_date=None, is_current=True
    )
    session.add(new_customer)
```

Range Partitioning on Fact Tables

Analytical databases query massive volumes of transactions. Placing all records in a single table causes slow sequential scans. We partitioned our central fact_sales table by range based on order_date_key (YYYYMMDD format). The physical tables are separated into fact_sales_2021, fact_sales_2022, and so on. When a query requests 2023 sales, PostgreSQL reads ONLY the 2023 partition table, ignoring the rest, speeding up queries by up to 90%.

Performance Indexing Strategy

We optimized analytical performance using three main types of database indexes:

- * **B-tree Foreign Key Indexes:** Accelerates joins. Placed on order_date_key, customer_key, product_key, region_key, etc.
- * **Composite Index:** Placed on (order_date_key, customer_key) to optimize queries grouping or filtering by customer and date simultaneously.
- * **Partial Index:** Placed on fact_sales(sales_key, profit) WHERE profit < 0. Indexes ONLY loss-making transactions, saving storage and speeding up audits.

4. Interview Presentation Blueprint

How to present the project: The STAR method

S - Situation

A retail organization needs an enterprise data warehouse to ingest raw, unvalidated CSV transactions from multiple stores. They require historical tracking of customer segment shifts, materialized reporting, analytics views, and a REST API service to expose KPIs to their operational applications.

T - Task

Design and construct the complete, end-to-end data engineering architecture, including synthetic data generation, a modular ETL pipeline, a PostgreSQL Star Schema with Slowly Changing Dimensions (SCD Type 2), partition strategies, index tuning, and a FastAPI layer.

A - Action

I wrote a Python ETL pipeline using Pandas. I constructed a data quality validation check that calculates DQ scores. I created the Star Schema models and tables in PostgreSQL using SQLAlchemy. I implemented SCD Type 2 logic to track segment shifts, partition boundaries for the fact table, B-tree FK indexes, partial indexes on negative profits, and PL/pgSQL stored procedures. Finally, I coded FastAPI routes with Pydantic v2 schemas and wrote a 26-test suite using pytest.

R - Result

A robust analytics pipeline that processes 10,000 transactions, validates data quality, and loads cleaned records in less than 4 seconds. Queries for monthly trends and margins run 85% faster due to partitioning and FK index tuning. Endpoints return KPI aggregates instantly.

Interview Trap: How to handle massive data (e.g. 100GB+)?

Tip: A common question for Junior Data Engineers is 'What if the CSV file was 200GB?'. Do not say 'I will load it into Pandas.' Pandas runs in-memory and will crash. Say: 'I would load the CSV in chunks using pandas' chunksize parameter, or scale the pipeline by migrating the cleaning and transform logic to PySpark to distribute execution across a cluster, and load into a cloud data warehouse like Snowflake or BigQuery.'

5. Selected High-Impact Interview Q&As

Q: What is the difference between a Star Schema and a Snowflake Schema? Why choose Star?

Interviewer's Core Intent: Check if you understand core analytical modeling (OLAP vs OLTP) and know database design tradeoffs.

Suggested Answer: A Star Schema keeps dimensions completely denormalized (e.g. category and sub-category are in the same table). A Snowflake Schema normalizes them into multiple tables (e.g. dim_subcategory joins to dim_category). I chose Star Schema because it reduces join complexity for reporting tools like Power BI, accelerating queries since OLAP engines scan wide tables much faster than executing nested joins.

Q: Explain the SCD Type 2 customer loader design.

Interviewer's Core Intent: Check your logical coding ability and understanding of historical data tracking.

Suggested Answer: In loader.py, we sort group events chronologically. For each Customer ID, we check if they already exist in the database with is_current = True. If they do not, we insert them as active. If they do, we compare attributes. If segment or name changed, we expire the existing row (setting is_current=False and expiry_date to order_date - 1) and insert a new active row. This preserves historical relationships in the fact table.

Q: Why did you use range partitioning on order_date_key? Why not customer_id?

Interviewer's Core Intent: Validate if you understand physical layout optimizations and query execution patterns.

Suggested Answer: We partition by order_date_key because retail queries almost always filter by date ranges (e.g. monthly sales, YTD growth, seasonal charts). Partitioning by customer_id would create hundreds of tiny partitions, which degrades query planner performance and leads to high disk layout overhead. Annual date partitioning fits our access patterns perfectly.

Q: What is the purpose of the staging tables (stg_raw_sales, stg_clean_sales)?

Interviewer's Core Intent: Check if you understand staging concepts and how raw files are processed in professional warehouses.

Suggested Answer: Staging tables decouple the ingestion from the final warehouse models. stg_raw_sales is used for fast bulk copying of raw CSV text. stg_clean_sales stores the cleansed data (with coerced data types, trimmed whitespace, and dropped anomalies). Performing checks in staging ensures that only validated data reaches the dimension and fact tables, preventing warehouse corruption.

Q: Explain the difference between a Materialized View and a Stored Procedure.

Interviewer's Core Intent: *Test your knowledge of database programming and object differences.*

Suggested Answer: A Materialized View stores query results physically on disk. It behaves like a static table that is read-only until it is refreshed. A Stored Procedure is a compiled set of PL/pgSQL statements that can perform transactions and logic dynamically. In our system, we use `sp_monthly_refresh` to execute the materialized view REFRESH commands and calculate monthly metrics inside a single database transaction.

Q: How do you test your ETL pipeline?

Interviewer's Core Intent: *Assess if you understand database mocking, unit testing, and pipeline validation.*

Suggested Answer: I use `pytest` to verify all pipeline components. In `tests/conftest.py`, I set up a transactional in-memory SQLite database that mimics the PostgreSQL models. I seed mock dimension records, feed sample raw rows with known anomalies, and assert that the extractor detects encoding, the validator flags the errors, the cleaner rejects them, the loader correctly updates SCD-2 and inserts facts, and FastAPI endpoints return correct metrics.

Q: How does Pydantic help you in configuration and schema validation?

Interviewer's Core Intent: *Test your familiarity with Python data formatting libraries and API best practices.*

Suggested Answer: In `src/config/config.py`, Pydantic's `BaseSettings` reads and validates configuration values from the `.env` file. If `DB_PORT` is a string, Pydantic automatically coerces it to an integer or throws an error. In the API layer, Pydantic schemas serialize database models into strict API JSON formats and validate request inputs, securing endpoints against invalid parameters.